

Principles of Distributed Systems

第9周：安全
Week 9: Security

By: Fahad Sabah

计算机学院,
北京工业大学, 中国
College of Computer Science,
Beijing University of Technology, China

fahad.sabah@bjut.edu.cn



安全

Security

❑ Introduction

❑ Secure channels

❑ Access control

❑ Security management

❑ 简介

❑ 安全通道

❑ 访问控制

❑ 安全管理

简介

Introduction

Basics

A component provides services to clients. To provide services, the component may require the services from other components) a component may depend on some other component.

基础

组件为客户提供服务。为了提供服务，组件可能需要其他组件的服务。) 组件可能依赖于其他组件。

Property	Description
Availability	Accessible and usable upon demand for authorized entities
Reliability	Continuity of service delivery
Safety	Very low probability of disasters
Confidentiality	No unauthorized disclosure of information
Integrity	No accidental or malicious alterations of information have been performed (even by authorized entities)

简介

Introduction

Observation:

In distributed systems, security is the combination of availability, integrity, and confidentiality. A dependable distributed system is thus fault tolerant and secure.

The Players

Subject: Entity capable of issuing a request for a service as provided by objects

Channel: The carrier of requests and replies for services offered to subjects

Object: Entity providing services to subjects.

观察:

在分布式系统中，安全是可用性、完整性和机密性的结合。因此，一个可靠的分布式系统具有容错性和安全性。

玩家

主题: 能够根据对象提供的服务请求发出请求的实体

通道: 向主体提供服务请求和回复的载体

对象: 向主体提供服务的实体。

简介

Introduction

Security threats [安全威胁]

Threat	Channel	Object
Interruption	Preventing message transfer	Denial of service
Inspection	Reading the content of transferred Messages	Reading the data contained in an object
Modification	Changing message content	Changing an object's encapsulated data
Fabrication	Inserting messages	Spoofing an object

简介

Introduction

Security mechanisms

To protect against security threats, we have a number of security mechanisms at our disposal:

1. **Encryption:** Transform data into something that an attacker cannot understand (confidentiality). It is also used to check whether something has been modified (integrity).
2. **Authentication:** Verify the claim that a subject says it is S: verifying the identity of a subject.
3. **Authorization:** Determining whether a subject is permitted to make use of certain services.
4. **Auditing:** Trace which subjects accessed what, and in which way. Useful only if it can

安全机制

为了防范安全威胁，我们拥有多种安全机制：

1. **加密：**将数据转化为攻击者无法理解的（保密性）。它也用于检查某物是否被修改（完整性）。
2. **认证：**验证主体自称是S的主体：验证主体身份。
3. **授权：**确定主体是否被允许使用某些服务。
4. **审计：**追踪哪些受试者访问了什么，以及以何种方式访问。只有在能帮助抓住攻击者时才有用。

简介

Introduction

Security policies

Prescribes how to use mechanisms to protect against attacks.

Requires that a model of possible attacks is described (i.e., security architecture).

安全政策

规定如何使用机制来防御攻击。
需要描述可能攻击的模型（即安全架构）。

Think of security policies like rules in a game or in your house that keep everyone safe.

- ☐ A security policy is a rulebook for how to protect things from bad people.
- ☐ It tells us what tools to use and how to use them.
- ☐ To make good rules, we first imagine what bad things could happen. That's called a “security model” or “security architecture.”

简介

Introduction

Globus Security Rules

Imagine a big playground with different groups of friends and grown-ups watching.

- ☐ There are many groups (administrative domains), each with their own rules.
- ☐ When someone does something inside their own group, they follow their group's rules.
- ☐ If someone wants to do something in the whole playground, they must be known by everyone first.
- ☐ When groups talk to each other, they must prove who they are (mutual authentication).
- ☐ If someone is allowed in the whole playground, that can replace the smaller group's rules sometimes.
- ☐ A person can let a helper (a process) do things for them if they trust them.
- ☐ Helpers in the same group can share the same “ticket” or pass (credentials) to work together.

简介

Introduction

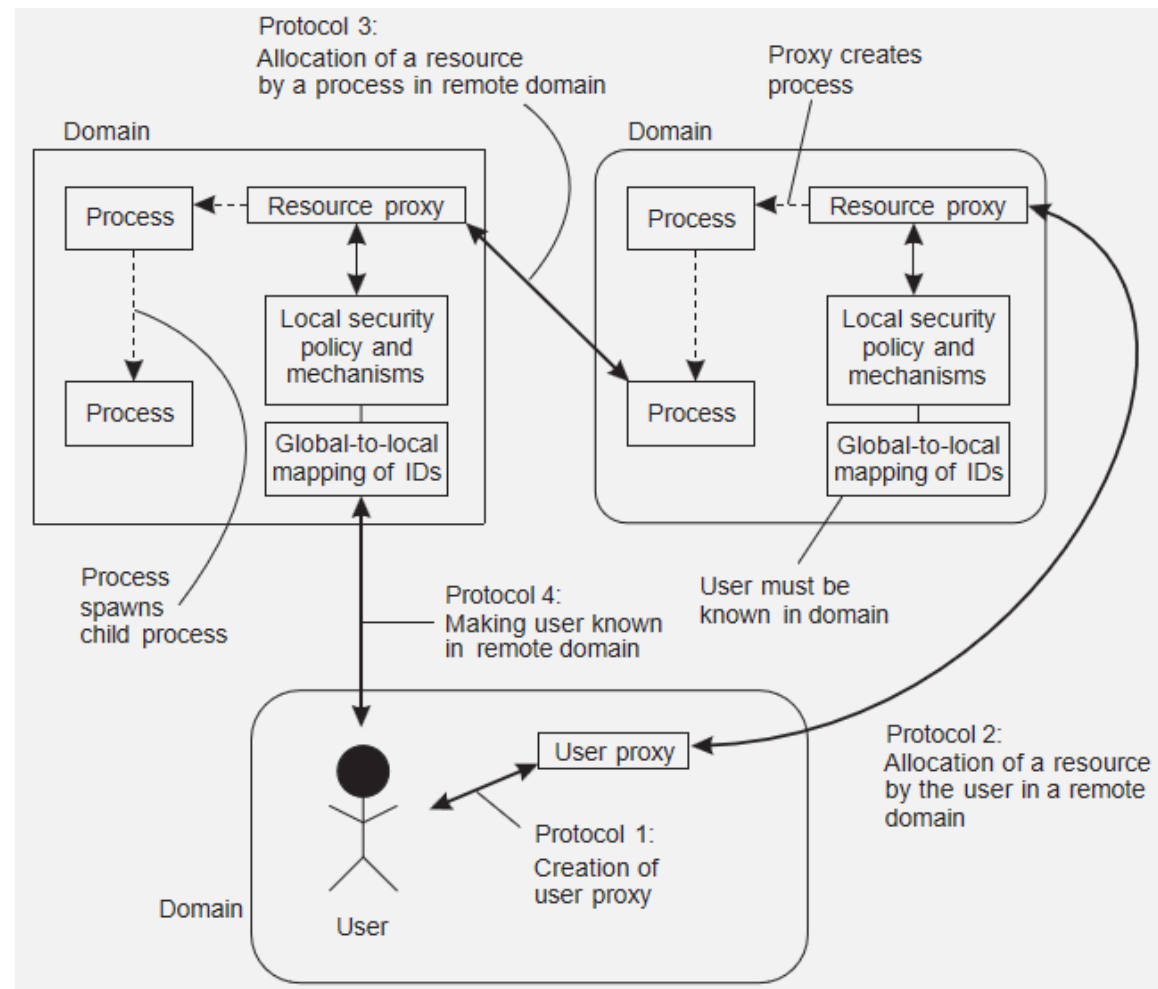
How two different computer groups (domains) work together safely.

Each domain has:

- ❑ a person or user
- ❑ a process that does work
- ❑ a local security guard (local security policy)
- ❑ a helper called a proxy

What happens

1. A user asks their domain to do something in another domain.
 - ❑ The user gets a “user proxy” as a helper.
2. The first domain creates a helper process to talk to the other domain.
 - ❑ That helper is a “resource proxy.”
3. The second domain checks:
 - ❑ who is asking
 - ❑ if they are allowed
 - ❑ if the user is known there



简介

Introduction

The four protocols in the picture

Protocol 1: The user makes a user proxy.

- ❑ It is like giving permission to a helper to act for the user.

Protocol 2: The user asks for a resource in a remote domain.

- ❑ The request is sent to the other side.

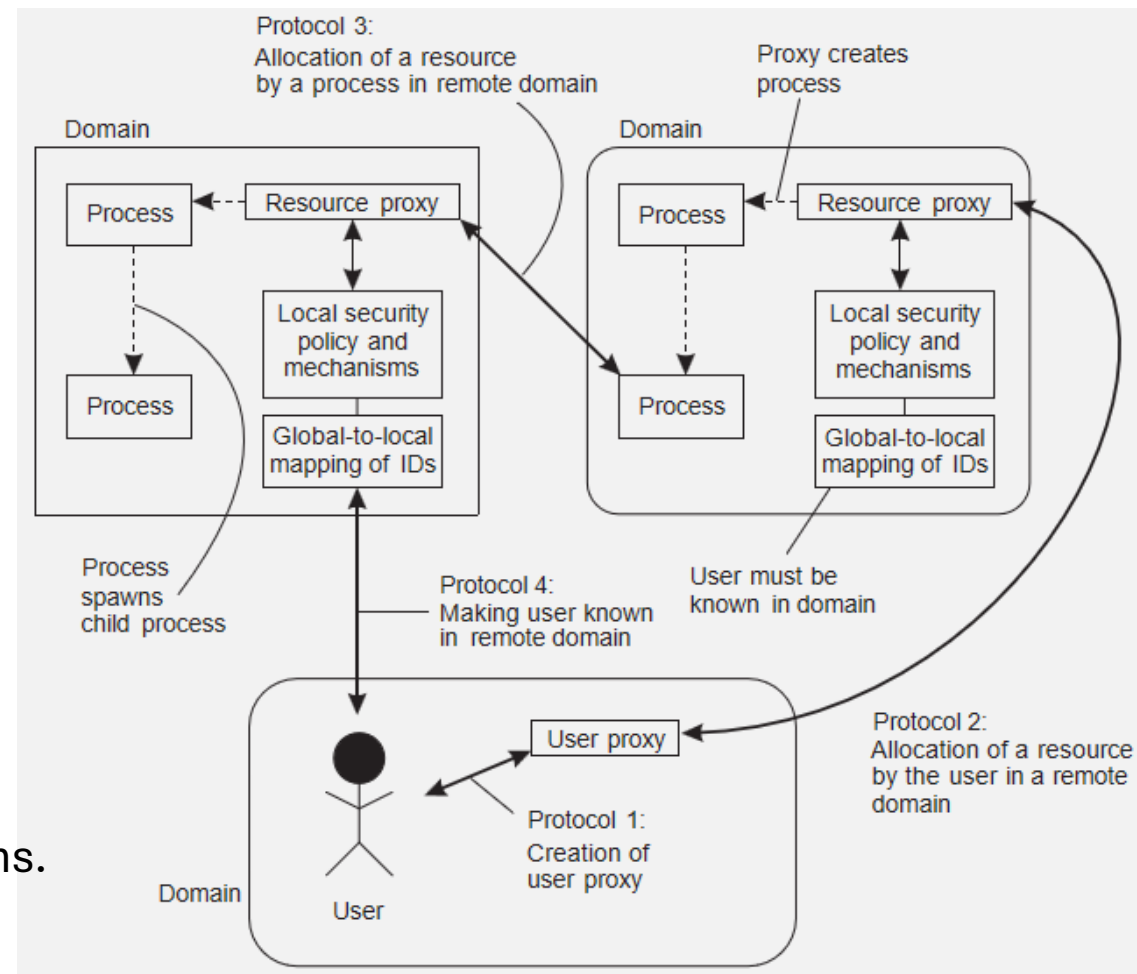
Protocol 3: A process in one domain asks for a resource in the other domain.

- ❑ The workplaces talk to each other.

Protocol 4: Making sure the user is known in the remote domain.

Key idea

- ❑ The other side checks the user is trusted.
- ❑ Each domain has its own local safety rules.
- ❑ When they work together, they must prove who they are.
- ❑ Helpers (proxies) carry the user's rights between domains.
- ❑ The remote domain must know the user before it allows work.

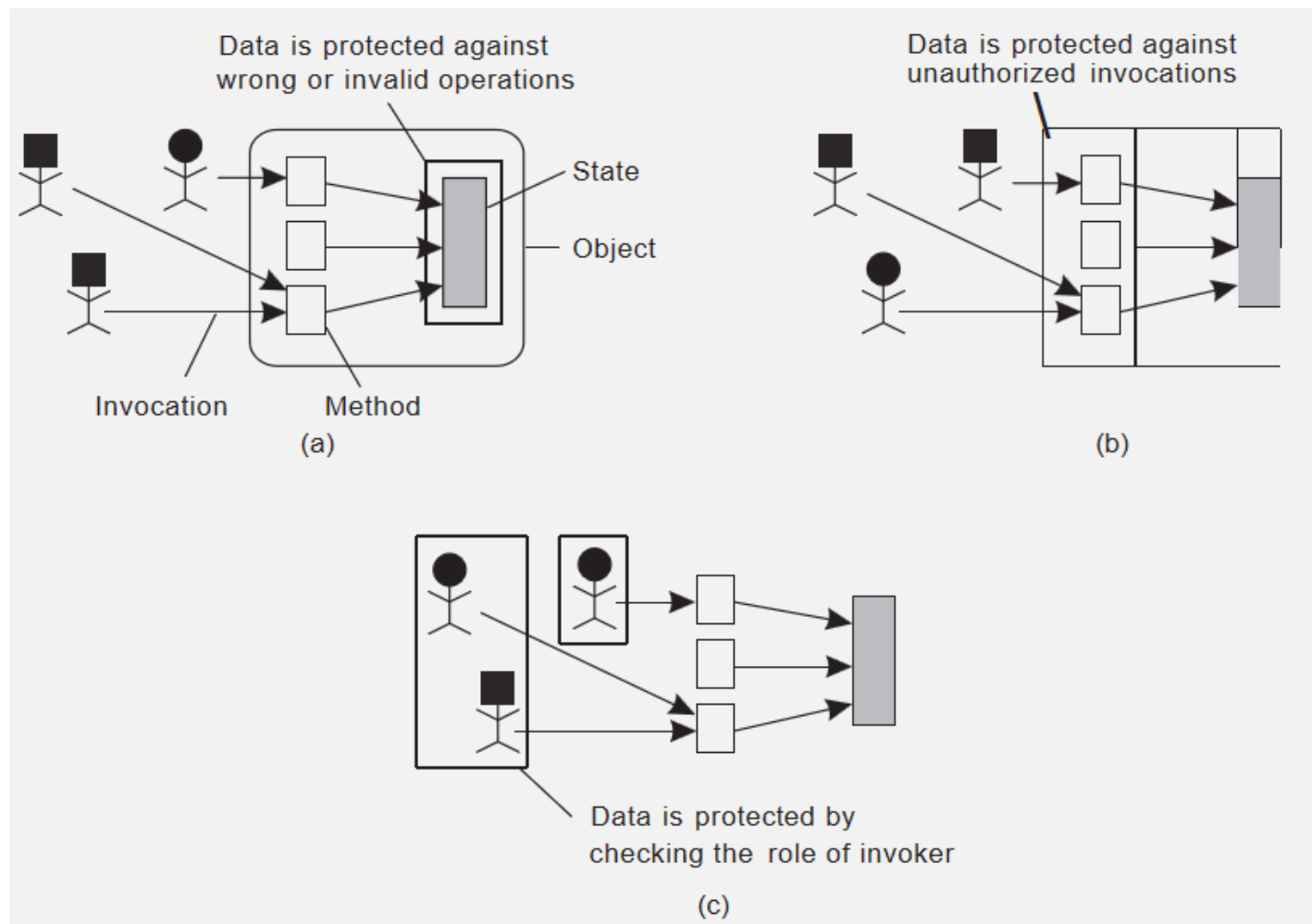


简介

Introduction

设计问题：控制焦点

Design issue: Focus of control



简介

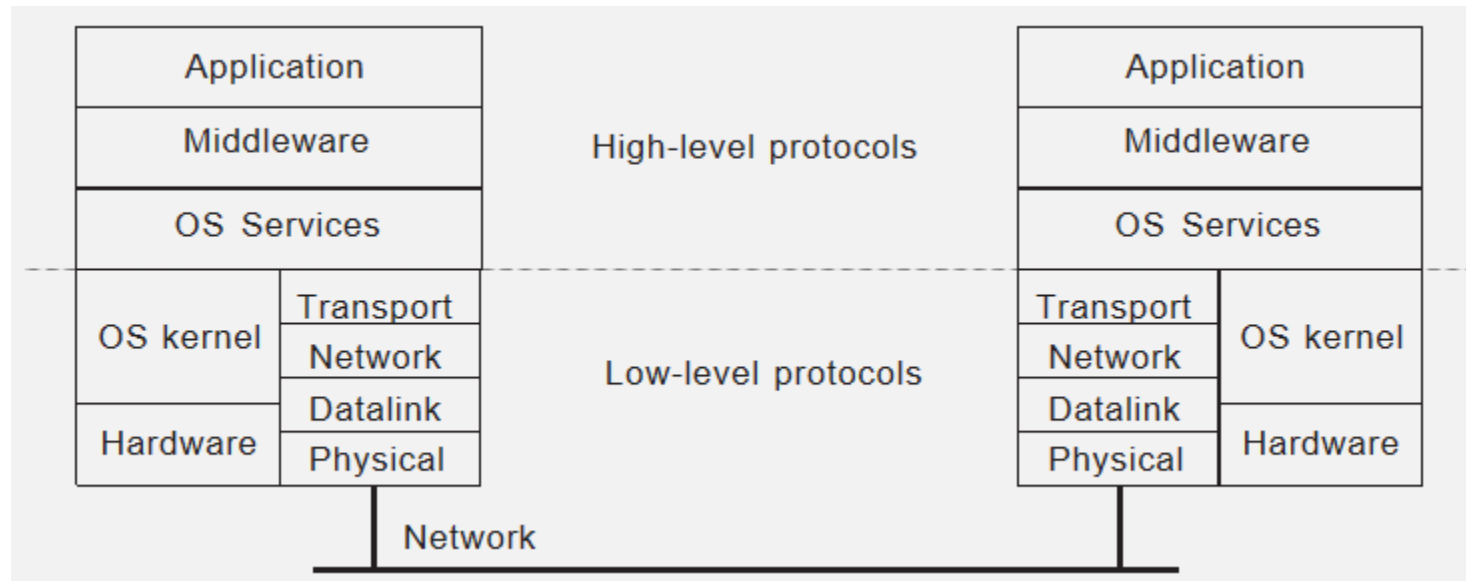
Introduction

Design issue: Layering of mechanisms and TCB

设计问题：机构分层与TCB

At which logical level are we going to implement security mechanisms?

我们将在哪个逻辑层面实施安全机制？



简介

Introduction

Design issue: Layering of mechanisms and TCB

Whether security mechanisms are actually used is related to the trust a user has in those mechanisms. No trust implement your own mechanisms.

Trusted Computing Base

What is the set of mechanisms needed to enforce a policy.

The smaller, the better.

设计问题：机构分层与TCB

是否真的使用安全机制，取决于用户对这些机制的信任程度。没有信任实施你自己的机制。

可信计算基础

执行一项政策所需的一套机制是什么？

越小越好。

简介

Introduction

Cryptography

1. Symmetric system

- ☐ One key does both jobs: lock and unlock.
- ☐ Both people must keep the same key secret.
- ☐ Like one shared password that both sender and receiver know.

2. Asymmetric system

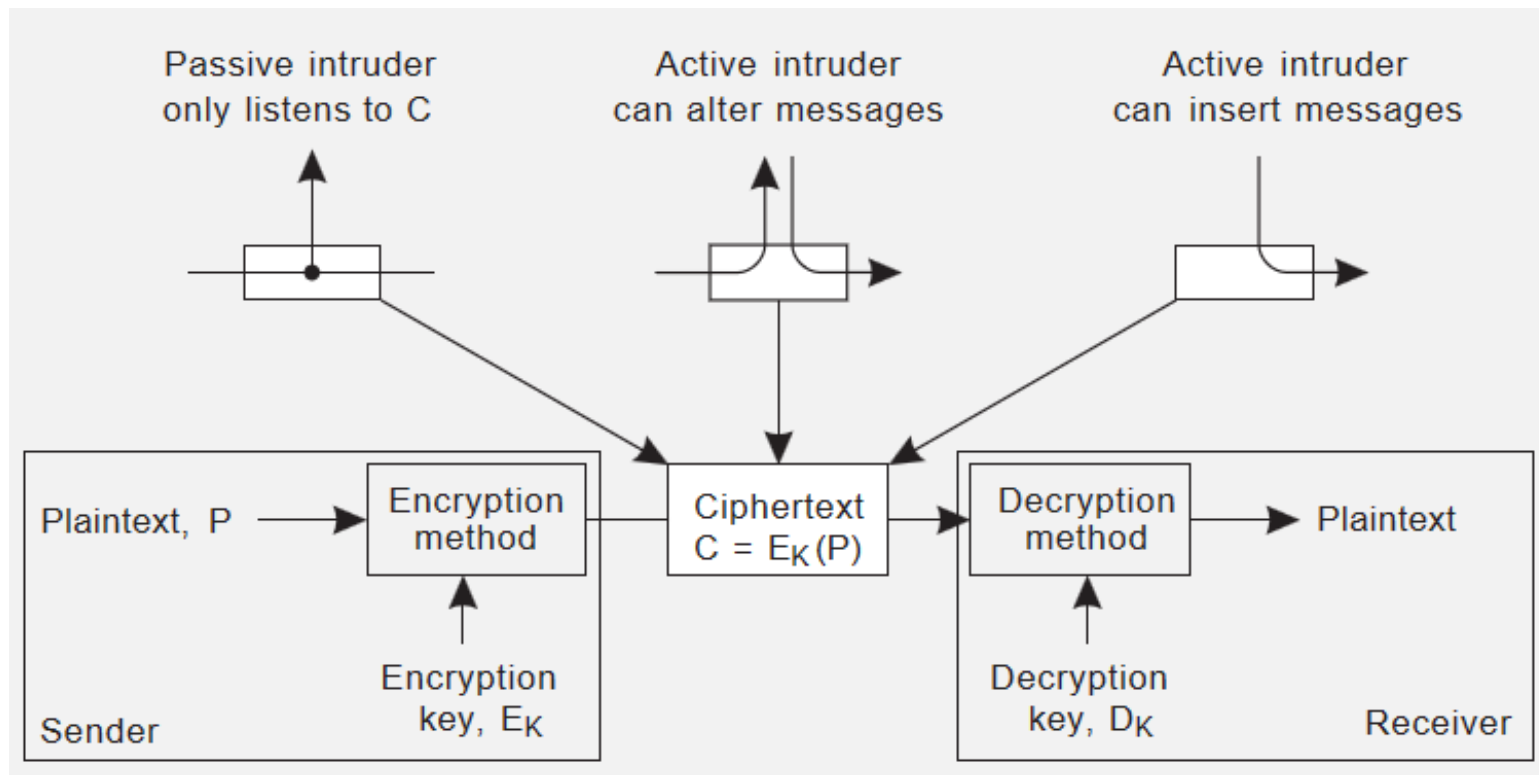
- ☐ Two keys are used:
 - ☐ One private key (kept secret)
 - ☐ One public key (shared with everyone)
 - ☐ One key locks the message, the other key unlocks it.
- ☐ It is like a mailbox: anyone can drop a letter in (public key), but only the owner can open it (private key).

3. Hashing system

- ☐ It turns data into a short fingerprint called a digest.
- ☐ There is no way to turn the digest back into the original message.
- ☐ It is used to check if the message is the same, not to hide it forever.

简介

Introduction



简介

Introduction

Cryptographic functions

1. One-way function: Given some output m_{out} of ES, it is (analytically or) computationally infeasible to find min : $ES(min) = m_{out}$

Example: Imagine a paper shredder:

You can put a paper in and get shredded bits.

But you cannot take the shredded bits and rebuild the original paper.

2. Weak collision resistance: Given the pair $\langle m; ES(m) \rangle$, it is computationally infeasible to find an $m^* \neq m$ such that $ES(m^*) = ES(m)$

Example: Like knowing one toy and its secret code, and still not being able to find another toy with the same code.

3. Strong collision resistance: It is computationally infeasible to find any two different inputs m^* and m such that $ES(m^*) = ES(m)$

Example: Like trying to find two different toys that accidentally share the exact same secret code.

安全通道

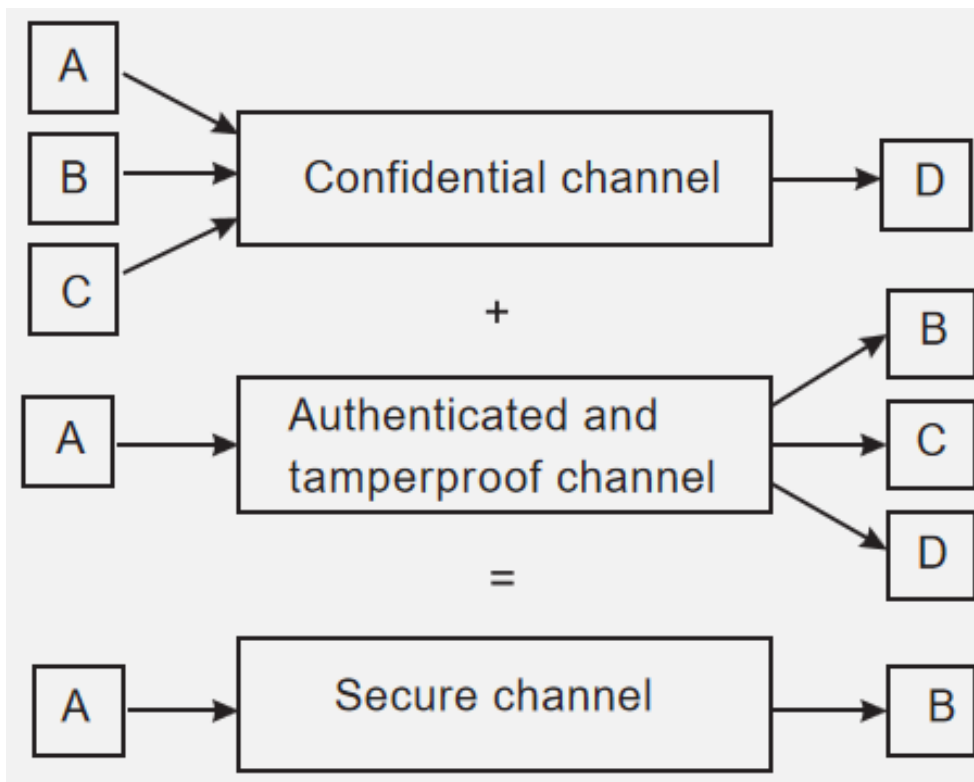
Secure channels

- ❑ Authentication
- ❑ Message Integrity and confidentiality
- ❑ Secure group communication

- ❑ 认证
- ❑ 消息完整性与机密性
- ❑ 安全组通信

安全通道

Secure channels



- ❑ 双方都知道对方是谁（已认证）。
- ❑ 双方都知道消息不能被篡改（完整性）。
- ❑ 双方都知道信息不能泄露（保密性）。

- ❑ Both parties know who is on the other side (authenticated).
- ❑ Both parties know that messages cannot be tampered with (integrity).
- ❑ Both parties know messages cannot leak away (confidentiality).

Week 9 Security 第9周：安全

安全通道

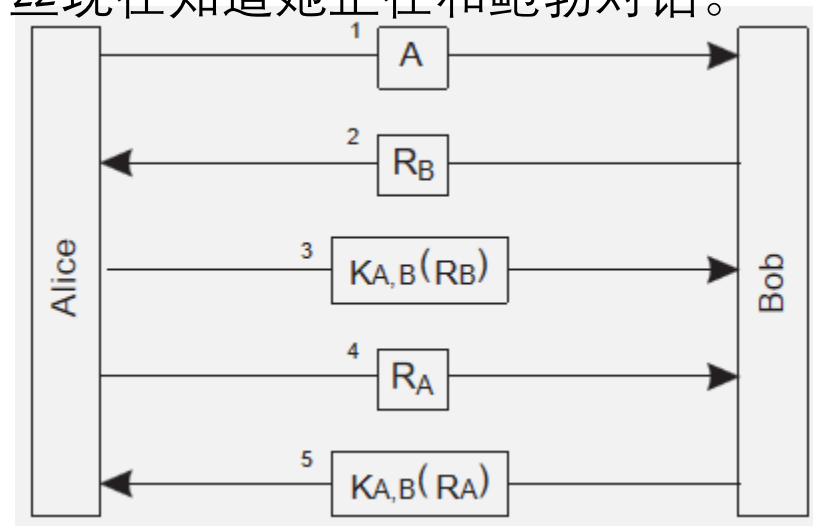
Secure channels

认证：秘密密钥

- 1: 爱丽丝向鲍勃发送身份证
- 2: 鲍勃向爱丽丝发出挑战 R_B
- 3: 爱丽丝用共享密钥 $K_{A,B}$ 加密了 R_B 。鲍勃现在知道自己正在和爱丽丝通信。
- 4: 爱丽丝向鲍勃发挑战 R_A
- 5: 鲍勃用 $K_{A,B}$ 加密了 R_A 。爱丽丝现在知道她正在和鲍勃对话。

Authentication: Secret keys

- 1: Alice sends ID to Bob
- 2: Bob sends challenge R_B to Alice
- 3: Alice encrypts R_B with shared key $K_{A,B}$. Bob now knows he is talking to Alice.
- 4: Alice sends challenge R_A to Bob
- 5: Bob encrypts R_A with $K_{A,B}$. Alice now knows that she is talking to Bob.



安全通道

Secure channels

认证：秘密密钥

改进

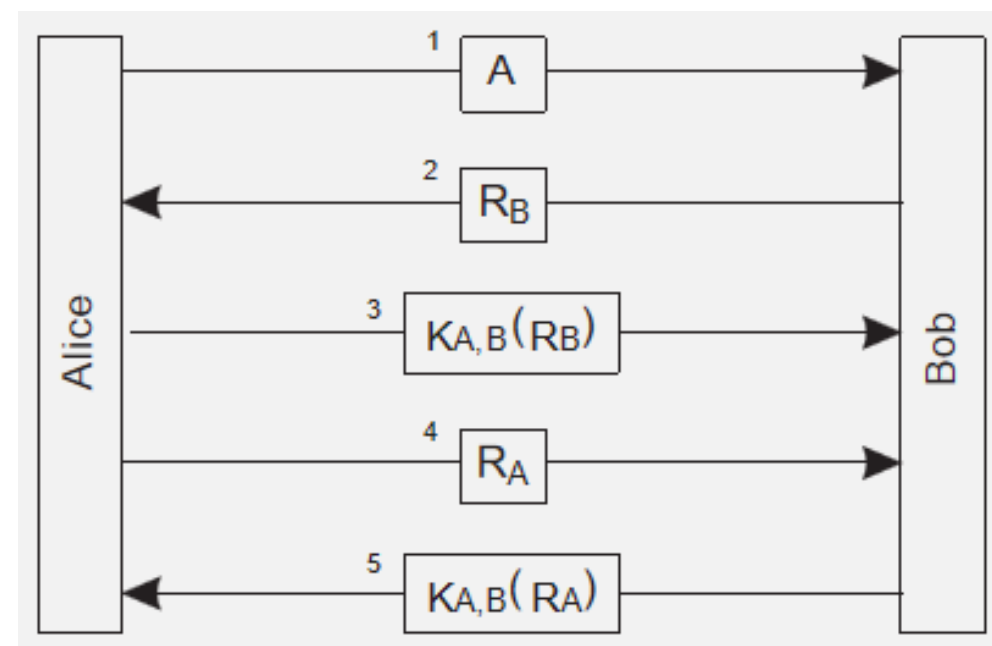
将步骤1和4和2、5合并。代价：正确性。

Authentication: Secret keys

Improvement

Combine steps 1&4, and 2&5.

Price to pay: correctness.



安全通道

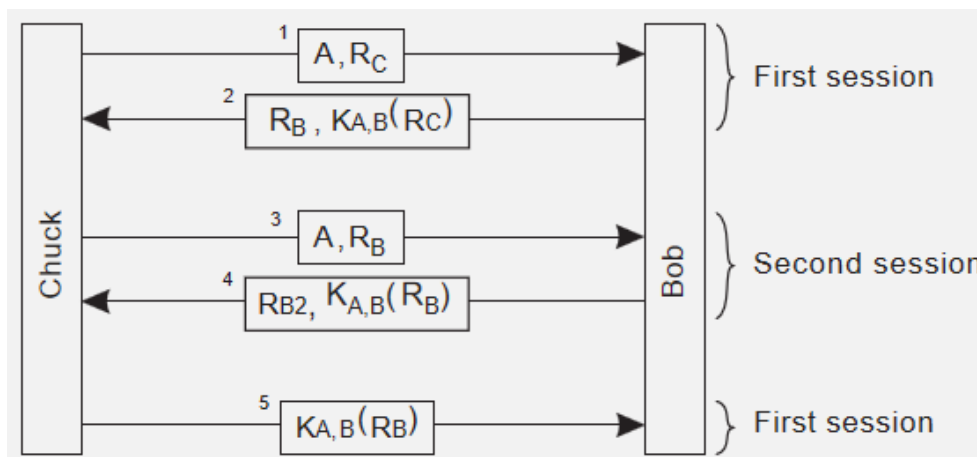
Secure channels

Authentication: Secret keys reflection attack

- 1: Chuck claims he's Alice, and sends challenge R_C
- 2: Bob returns a challenge R_B and the encrypted R_C
- 3: Chuck starts a second session, claiming he is Alice, but uses challenge R_B
- 4: Bob sends back a challenge, plus $K_{A,B}(R_B)$
- 5: Chuck sends back $K_{A,B}(R_B)$ for the first session to prove he is Alice.

认证：秘密密钥反射攻击

- 1: 查克声称自己是爱丽丝，并向 R_C 发出挑战
- 2: 鲍勃返回挑战， R_B 和加密的 R_C
- 3: 查克开始第二场游戏，声称自己是爱丽丝，但使用挑战 R_B
- 4: 鲍勃发回挑战，加上 $K_{A,B}(R_B)$
- 5: 查克送回 $K_{A,B}(R_B)$ 用于第一次证明他是爱丽丝的游戏。



Week 9: Security

第9周：安全

北京工业大学 分布式系统的原理

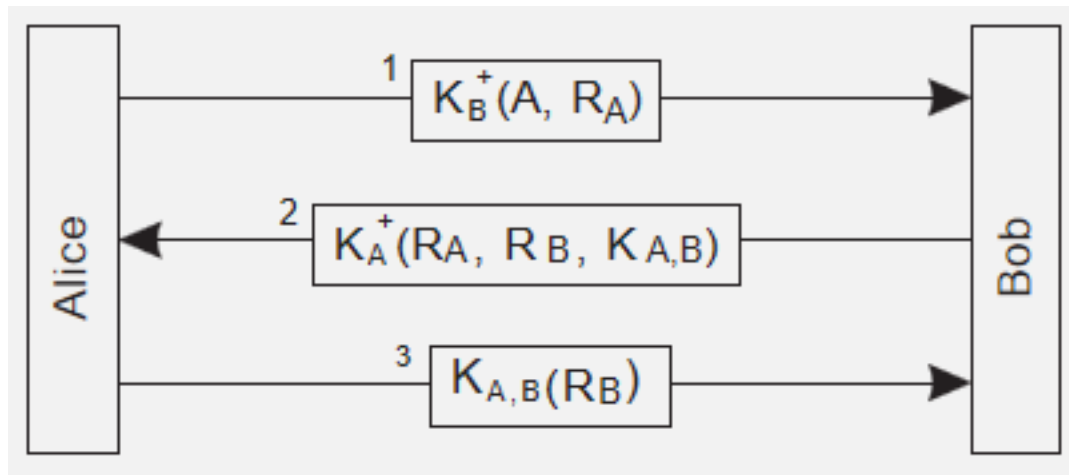
Beijing University of Technology, Principles of Distributed Systems

安全通道

Secure channels

Authentication: Public key

- 1: Alice sends a challenge R_A to Bob, encrypted with Bob's public key K_B^+ .
- 2: Bob decrypts the message, generates a secret key $K_{A,B}$ (session key), proves he's Bob (by sending R_A back), and sends a challenge R_B to Alice. Everything's encrypted with Alice's public key K_A^+ .
- 3: Alice proves she's Alice by sending back the decrypted challenge, encrypted with generated secret key $K_{A,B}$.

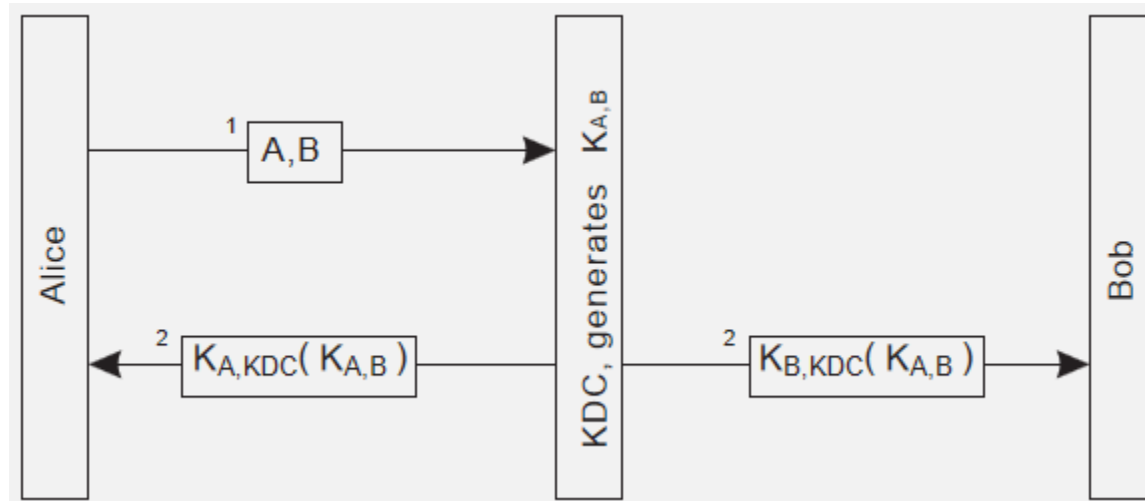


安全通道

Secure channels

Authentication: KDC

With N subjects, we need to manage $N(N - 1)/2$ keys, each subject knowing $N - 1$ keys) use a trusted Key Distribution Center that generates keys when necessary.



Question

How many keys do we need to manage?

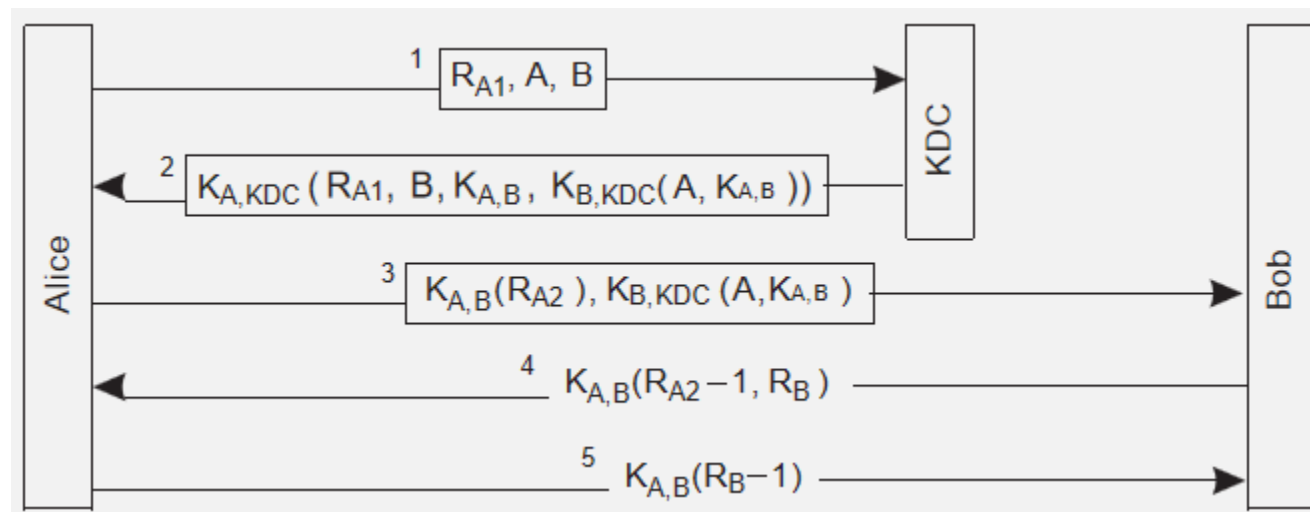
安全通道

Secure channels

Authentication: KDC (Needham-Schroeder)

Inconvenient

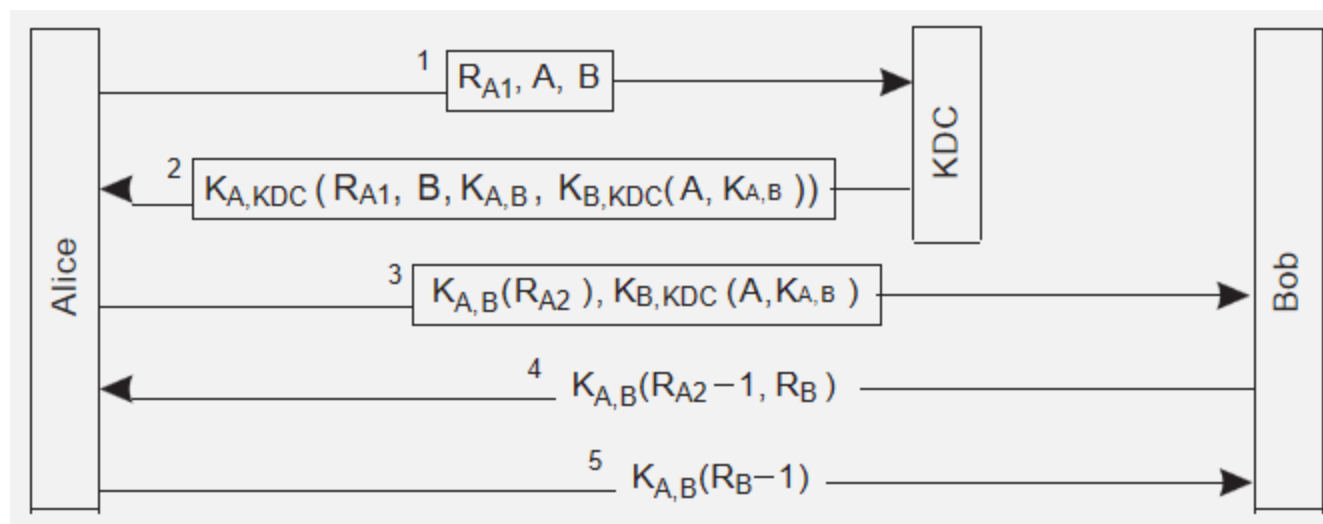
We need to ensure that Bob knows about $K_{A;B}$ before Alice gets in touch) let Alice do the work and pass her a ticket to set up a secure channel with Bob.



安全通道

Secure channels

Needham-Schroeder: Subtleties



Some issues

Q1 : Why does the KDC put Bob into its reply message, and Alice into the ticket?

Q2: The ticket sent back to Alice by the KDC is encrypted with Alice's key. Is this necessary?

安全通道

Secure channels

Needham-Schroeder: Subtleties

Security flaw

Suppose Trudy finds out Alice's key $\Rightarrow$ she can use that key anytime to impersonate Alice, even if Alice changes her private key at the KDC.

Reasoning

Once Trudy finds out Alice's key, she can use it to decrypt a (possibly old) ticket for a session with Bob, and convince Bob to talk to her using the old session key.

Solution

Have Alice get an encrypted number from Bob first, and put that number in the ticket provided by the KDC $\Rightarrow$ we're now ensuring that every session is known at the KDC.

安全通道

Secure channels

Confidentiality Solutions

Secret key: Use a shared secret key to encrypt and decrypt all messages sent between Alice and Bob

Public key: If Alice sends a message m to Bob, she encrypts it with Bob's public key: $K_B^+(m)$

Problems with keys

Keys wear out: The more data is encrypted by a single key, the easier it becomes to find that key
⇒ don't use keys too often

Danger of replay: Using the same key for different communication sessions, permits old messages to be inserted in the current session ⇒ don't use keys for different sessions

安全通道

Secure channels

Confidentiality

Problems with keys

Compromised keys: If a key is compromised, you can never use it again.

Really bad if all communication between Alice and Bob is based on the same key over and over again $\Rightarrow$ don't use the same key for different things.

Temporary keys: Untrusted components may play along perhaps just once, but you would never want them to have knowledge about your really good key for all times $\Rightarrow$ make keys disposable

安全通道

Secure channels

Confidentiality

Essence

Don't use valuable and expensive keys for all communication, but only for authentication purposes.

Consequence

Introduce a “cheap” session key that is used only during one single conversation or connection (“cheap” also means efficient in encryption and decryption).

安全通道

Secure channels

Digital signatures

Harder requirements

- ❑ **Authentication:** Receiver can verify the claimed identity of the sender
- ❑ **Nonrepudiation:** The sender can later not deny that he/she sent the message
- ❑ **Integrity:** The message cannot be maliciously altered during, or after receipt

Solution

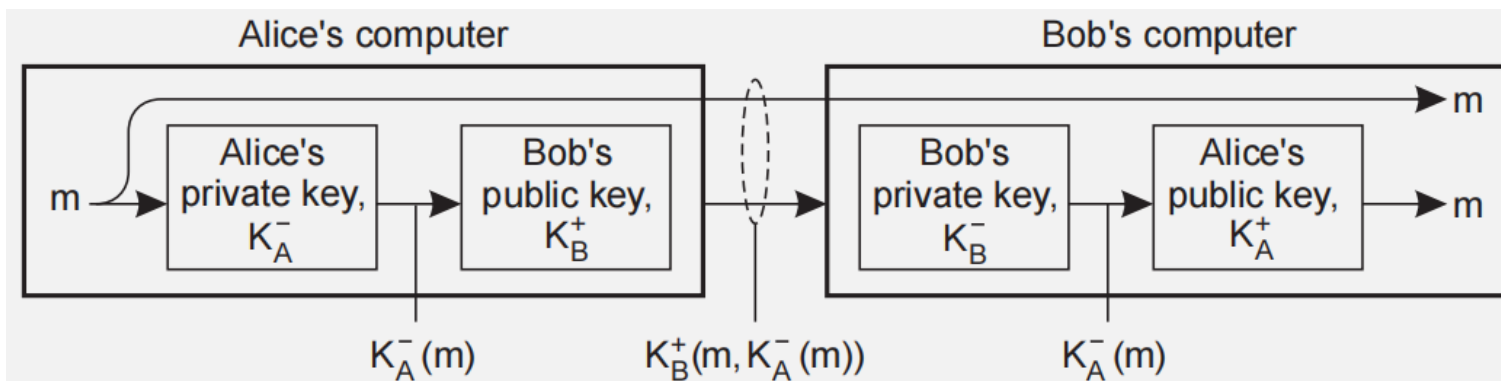
Let a sender sign all transmitted messages, in such a way that (1) the signature can be verified and (2) message and signature are uniquely associated

安全通道

Secure channels

Public key signatures

- 1: Alice encrypts her message m with her private key $K_A^- \Rightarrow m' K_A^-(m)$
- 2: She then encrypts m' with Bob's public key, along with the original message $m \Rightarrow m'' = K_B^-(m', K_A^-(m))$, and sends m'' to Bob.
- 3: Bob decrypts the incoming message with his private key K_B^- . We know for sure that no one else has been able to read m , nor m' during their transmission.
- 4: Bob decrypts m' with Alice's public key K_A^+ . Bob now knows the message came from Alice.



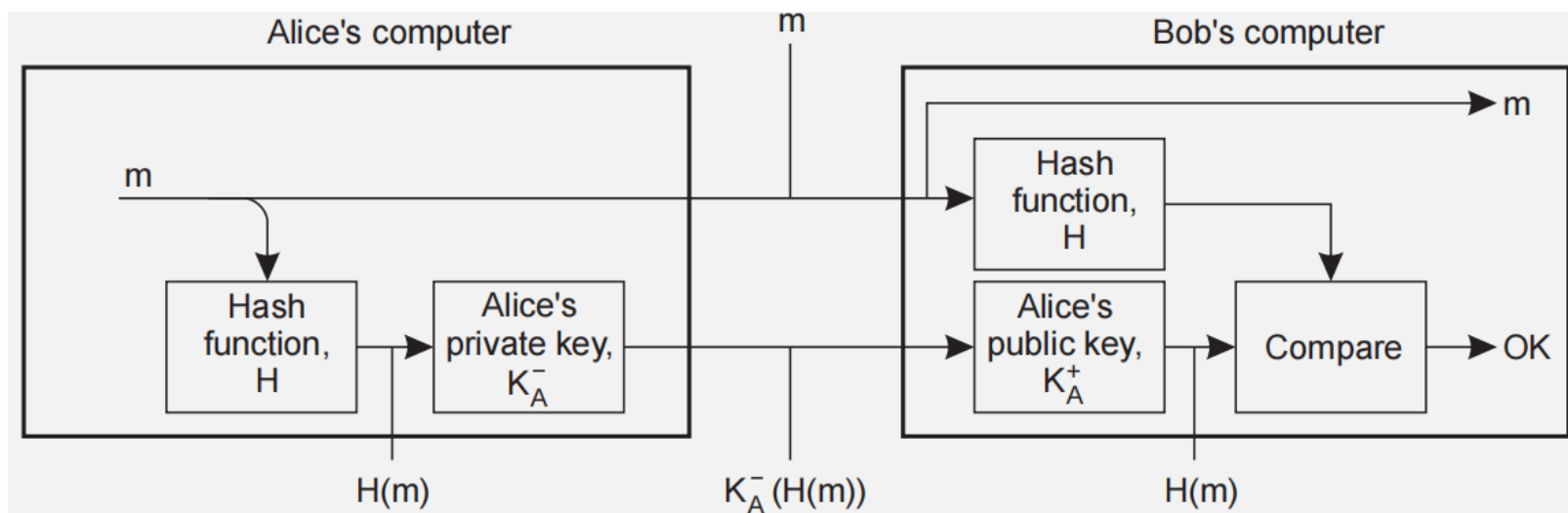
安全通道

Secure channels

Message digests

Basic idea

Don't mix authentication and secrecy. Instead, it should also be possible to send a message in the clear, but have it signed as well $\Rightarrow$ take a message digest, and sign that.



安全通道

Secure channels

Secure group communication

Design issue

How can you share secret information between multiple members without losing everything when one member turns bad.

Confidentiality

Follow a simple (hard-to-scale) approach by maintaining a separate secret key between each pair of members.

安全通道

Secure channels

Secure group communication

Replication

You also want to provide replication transparency. Apply secret sharing:

- ☐ No process knows the entire secret; it can be revealed only through joint cooperation
- ☐ Assumption: at most k out of N processes can produce an incorrect answer
- ☐ At most $c \leq k$ processes have been corrupted

Note

We are dealing with a k fault tolerant process group.

安全通道

Secure channels

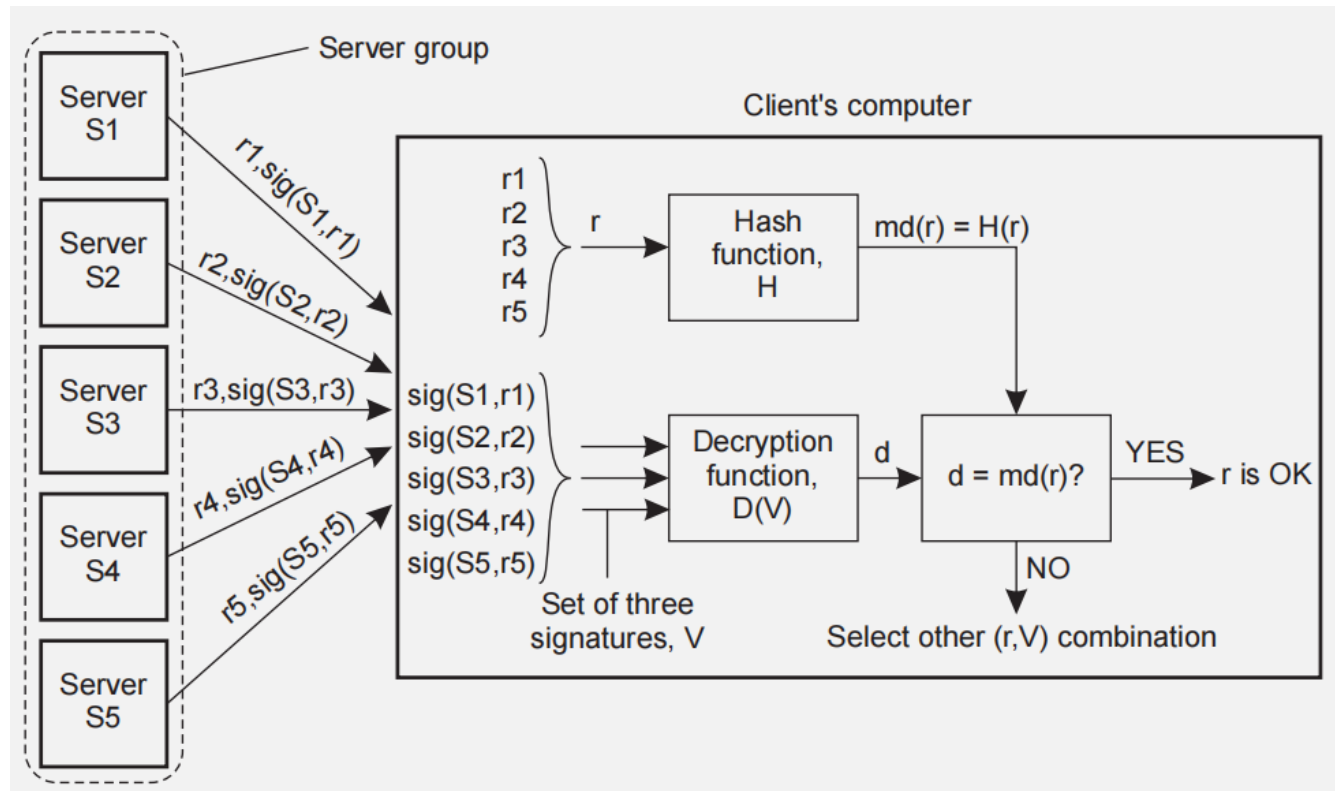
Secure replicated group

$N=5, c=2$

- Each server S_i sees each request and responds with r_i
- r_i is sent with digest $md(r_i)$, and signed with private key K_i^- .

Client uses special decryption function D that computes a single digest d from three signatures:

$$d = D(sig(S, r), sig(S' r'), sig(S'' r''))$$



If $d = md(r_i)$ for some r_i , r_i is considered correct

Also known as (m, n) -threshold scheme (with $m = c + 1, n = N$)

访问控制

Access control

- ☐ General issues
- ☐ Firewalls
- ☐ Secure mobile code

访问控制

Access control

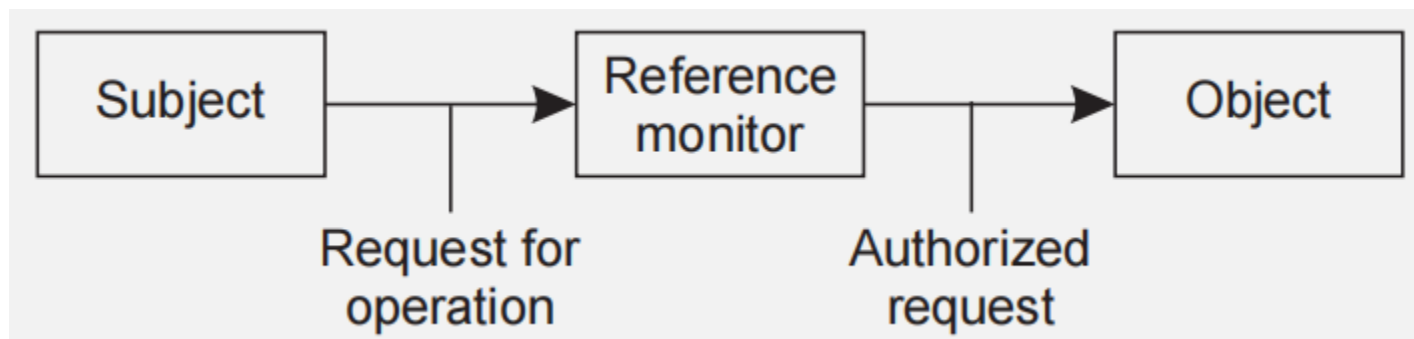
Authorization versus authentication

Authentication: Verify the claim that a subject says it is S: verifying the identity of a subject.

Authorization: Determining whether a subject is permitted certain services from an object.

Note

Authorization makes sense only if the requesting subject has been authenticated



访问控制

Access control

Access Control Matrix (ACM)

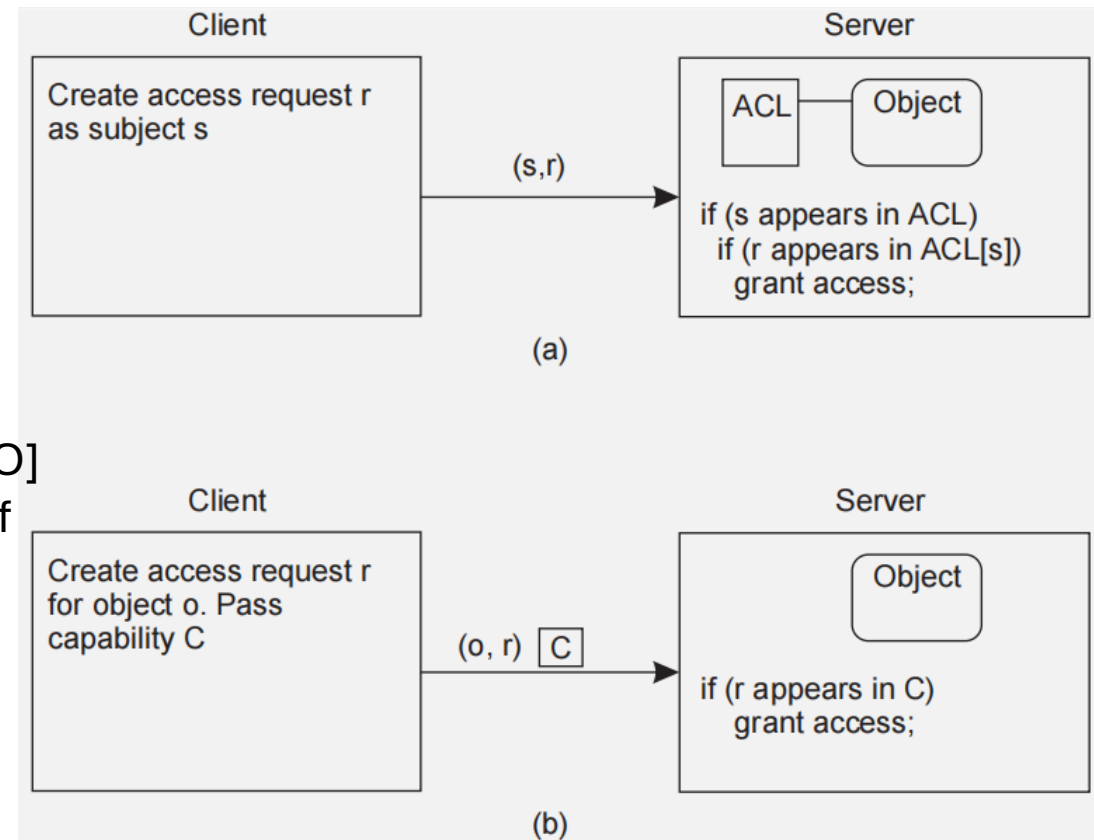
Maintain an access control matrix ACM in which entry $ACM[S,O]$ contains the permissible operations that subject S can perform on object O .

Access Control List (ACL)

Each object O maintains an access control list (ACL): $ACM[*,O]$ describing the permissible operations per subject (or group of subjects).

Capabilities

Each subject S has a capability: $ACM[S,*]$ describing the permissible operations per object (or category of objects).



访问控制

Access control

Protection domains

Issue

ACLs or capability lists can be very large. Reduce information by means of protection domains:

- ☐ Set of (object, access rights) pairs
- ☐ Each pair is associated with a protection domain
- ☐ For each incoming request the reference monitor first looks up the appropriate protection domain

Common implementation of protection domains

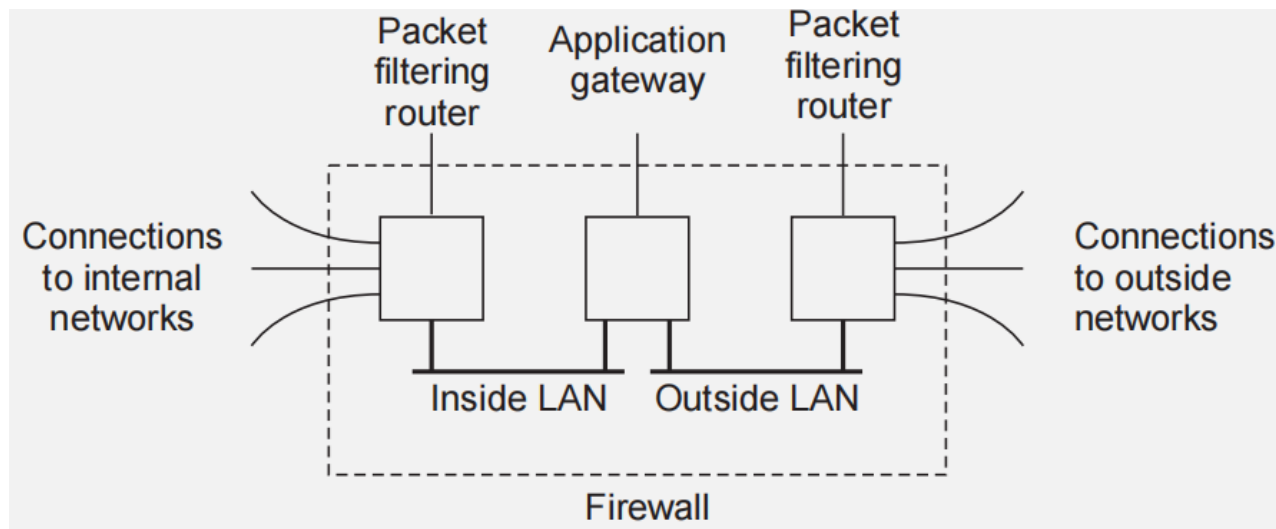
- ☐ Groups: Users belong to a specific group; each group has associated access rights
- ☐ Roles: Don't differentiate between users, but only the roles they can play. Your role is determined at login time. Role changes are allowed.

访问控制

Access control

Firewalls

Sometimes it's better to select service requests at the lowest level: network packets. Packets that do not fit certain requirements are simply removed from the channel $\Rightarrow$ protect by a firewall: it implements access control.



Question

What do you think would be the biggest breach in firewalls?

访问控制

Access control

Secure mobile code

Problem

Mobile code is great for balancing communication and computation, but:

- ❑ it is hard to implement a general-purpose mechanism that allows different security policies for local-resource access
- ❑ we may need to protect the mobile code (e.g., agents) against malicious hosts.

访问控制

Access control

Protecting an agent

Ajanta

Detect that an agent has been tampered with while it was on the move. Most important: append-only logs:

- ❑ Data can only be appended, not removed
- ❑ There is always an associated checksum. Initially, $C_{init} = K_{owner} + (N)$, with N a nonce.
- ❑ Adding data X by server S :

$$C_{new} = K_{owner} + (C_{old}, sig(S, X), S)$$

- ❑ Removing data from the log:

$$K_{owner}^{-}(C) \rightarrow C_{prev}, sig(S, X), S$$

allowing the owner to check integrity of X

访问控制

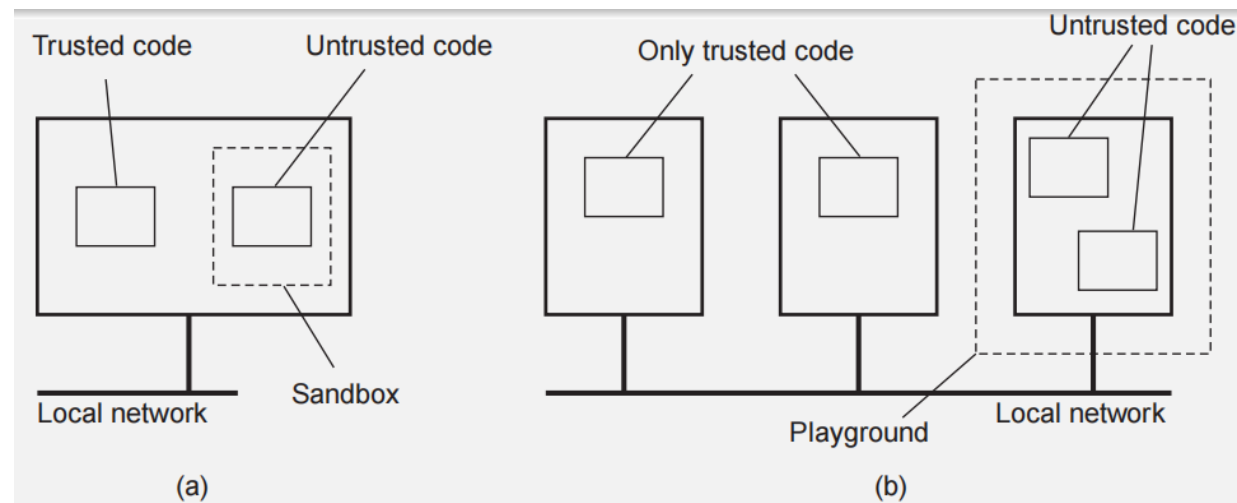
Access control

Protecting a host

Simple solution

Enforce a (very strict) single policy, and implement that by means of a few simple mechanisms:

- ❑ **Sandbox model:** Policy: Remote code is allowed access to only a pre-defined collection of resources and services. Mechanism: Check instructions for illegal memory access and service access
- ❑ **Playground model:** Same policy, but mechanism is to run code on separate “unprotected” machine.



访问控制

Access control

Protecting a host

Observation

We need to be able to distinguish local from remote code before being able to do anything.

Refinement 1

We need to be able to assign a set of permissions to mobile code before its execution and check operations against those permissions at all times

Refinement 2

We need to be able to assign different sets of permissions to different units of mobile code $\Rightarrow$ authenticate mobile code (e.g. through signatures)

Question

What would be a very simple policy to follow (Microsoft's approach)?

安全管理

Security management

- ☐ Key establishment and distribution
- ☐ Secure group management
- ☐ Authorization management

安全管理

Security management

Key establishment: Diffie-Hellman

Observation

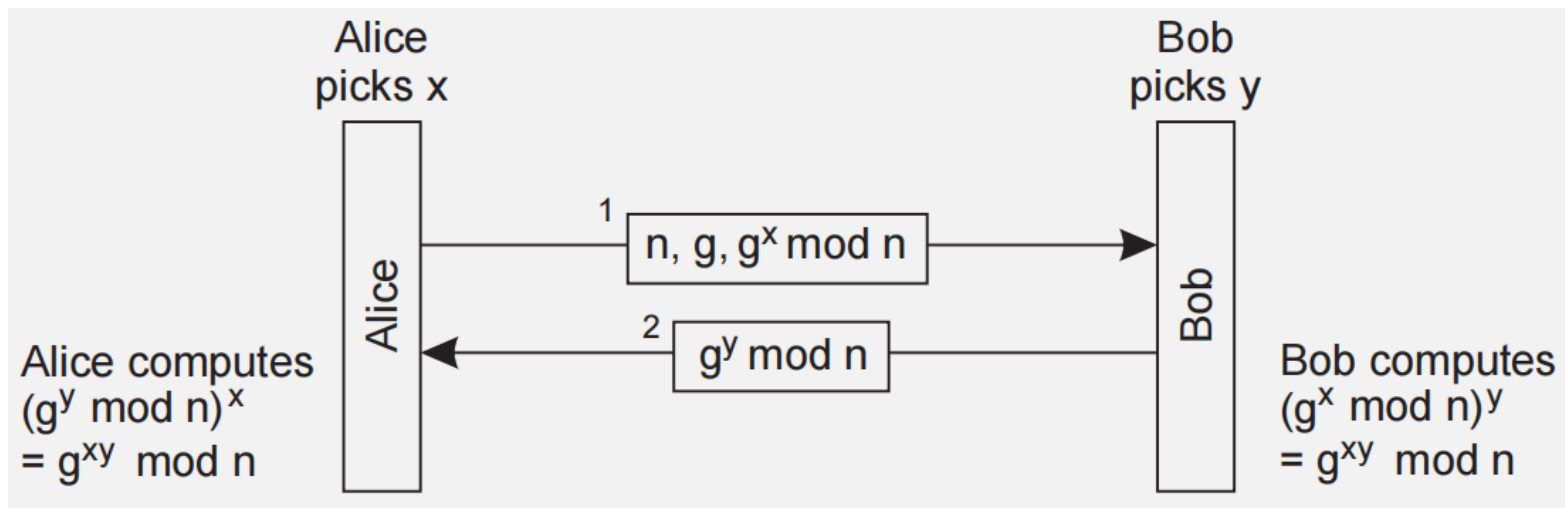
We can construct secret keys in a safe way without having to trust a third party (i.e. a KDC):

- ❑ Alice and Bob have to agree on two large numbers, n (prime) and g . Both numbers may be public.
- ❑ Alice chooses large number x , and keeps it to herself. Bob does the same, say y .

安全管理

Security management

Key establishment: Diffie-Hellman



1: Alice sends $(n, g, g^x \bmod n)$ to Bob

2: Bob sends $(g^y \bmod n)$ to Alice

3: Alice computes $K_{A,B} = (g^y \bmod n)^x = g^{xy} \bmod n$

4: Bob computes $K_{A,B} = (g^x \bmod n)^y = g^{xy} \bmod n$

Note

$n = kq + 1$, with q being prime > 160 bits. In practice, $n, g \in 512$ bits.

安全管理

Security management

Key distribution

Essence

If authentication is based on cryptographic protocols, and we need session keys to establish secure channels, who's responsible for handing out keys?

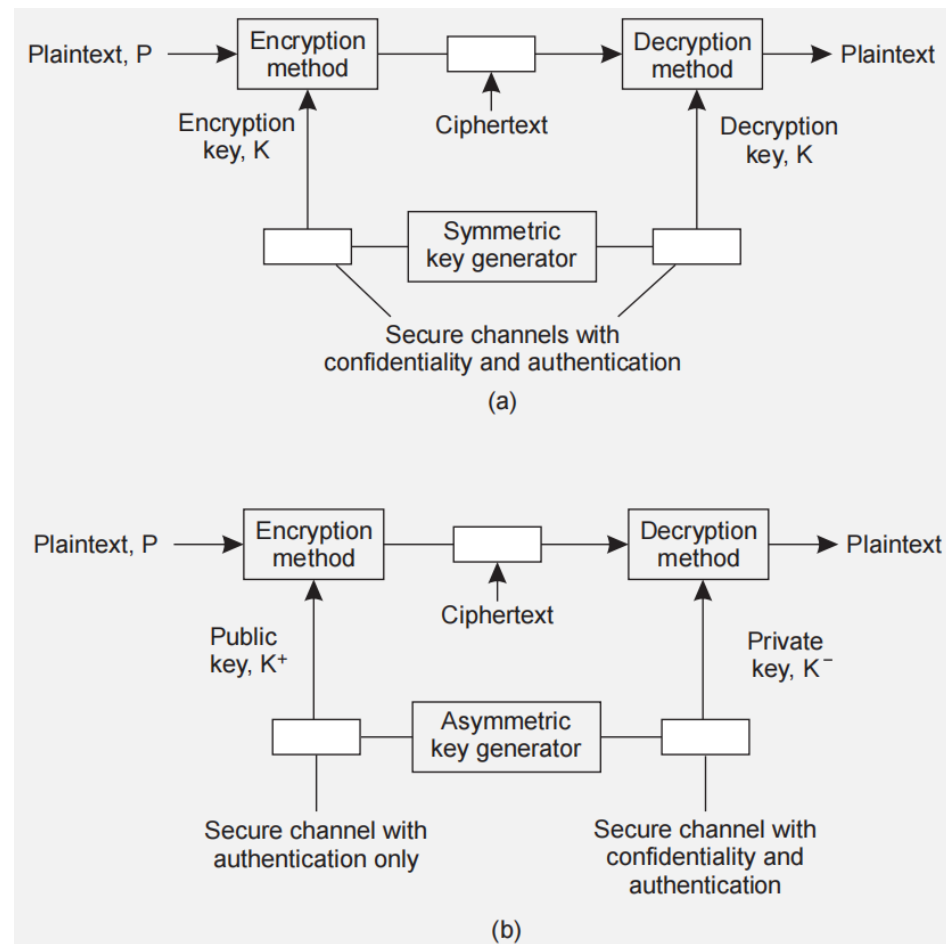
- ❑ **Secret keys:** Alice and Bob will have to get a shared key. They can invent their own and use it for data exchange. Alternatively, they can trust a key distribution center (KDC) and ask it for a key.
- ❑ **Public keys:** Alice will need Bob's public key to decrypt (signed) messages from Bob, or to send private messages to Bob. But she'll have to be sure about actually having Bob's public key, or she may be in big trouble. Use a trusted certification authority (CA) to hand out public keys.

A public key is put in a certificate, signed by a CA.

安全管理

Security management

Key distribution: getting keys to owners



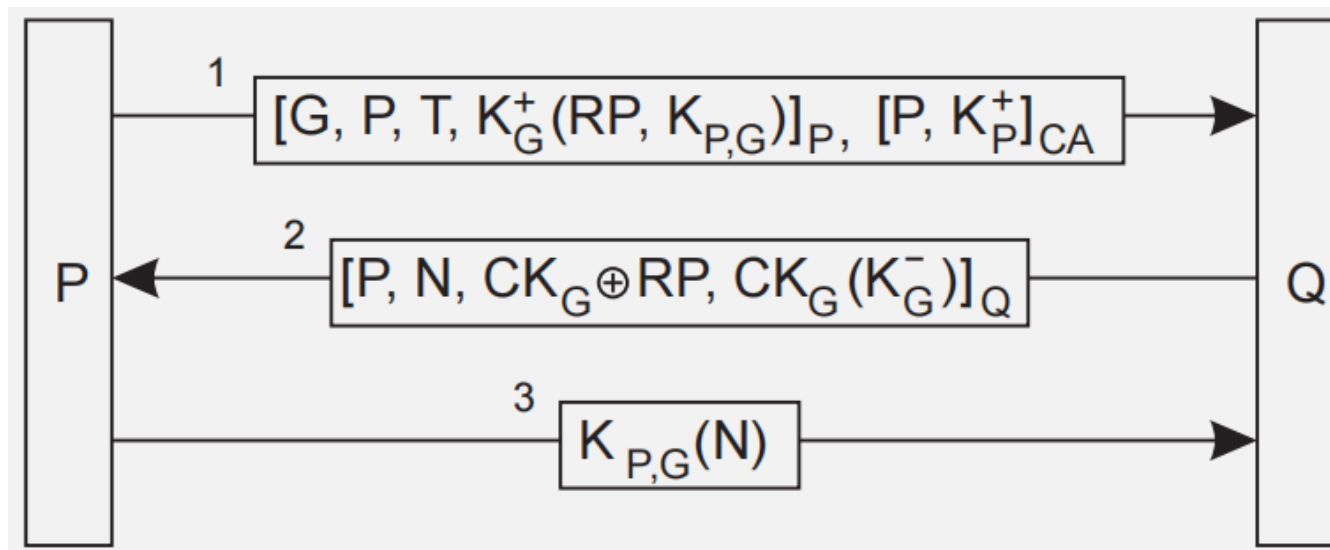
安全管理

Security management

Secure group management

Organization

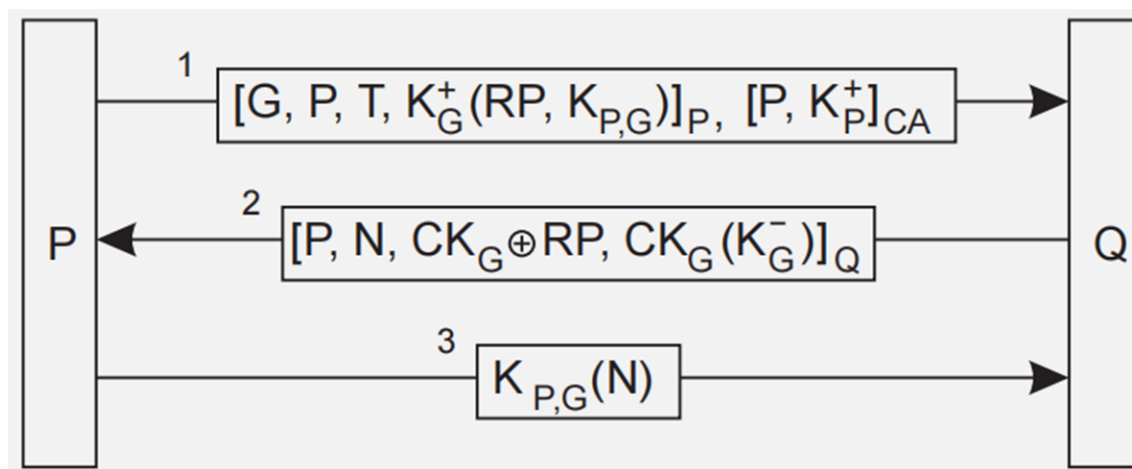
Group uses a key pair (K_G^+, K_G^-) for communication with nongroup members. There is a separate shared secret key CK_G for internal communication. Assume process P wants to join the group and contacts Q.



安全管理

Security management

Secure group management

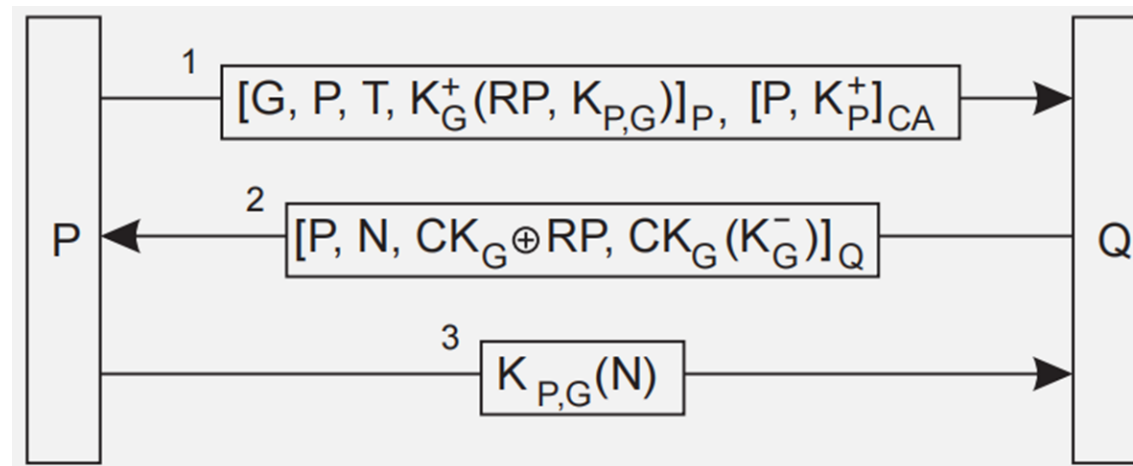


- 1: P generates a one-time reply pad RP , and a secret key $K_{P,G}$. It sends a join request to Q, signed by itself (notation: $[JR]_P$), along with a certificate containing its public key K_P^+ .
- 2: Q authenticates P, checks whether it can be allowed as member. It returns the group key CK_G , encrypted with the one-time pad, as well as the group's private key, encrypted as $CK_G(K_G^-)$.
- 3: Q authenticates P and sends back $K_{P,G}(N)$ letting Q know that it has all the necessary keys.

安全管理

Security management

Secure group management



Question

Why didn't we send $K_P^+(CK_G)$ instead of using RP ?

安全管理

Security management

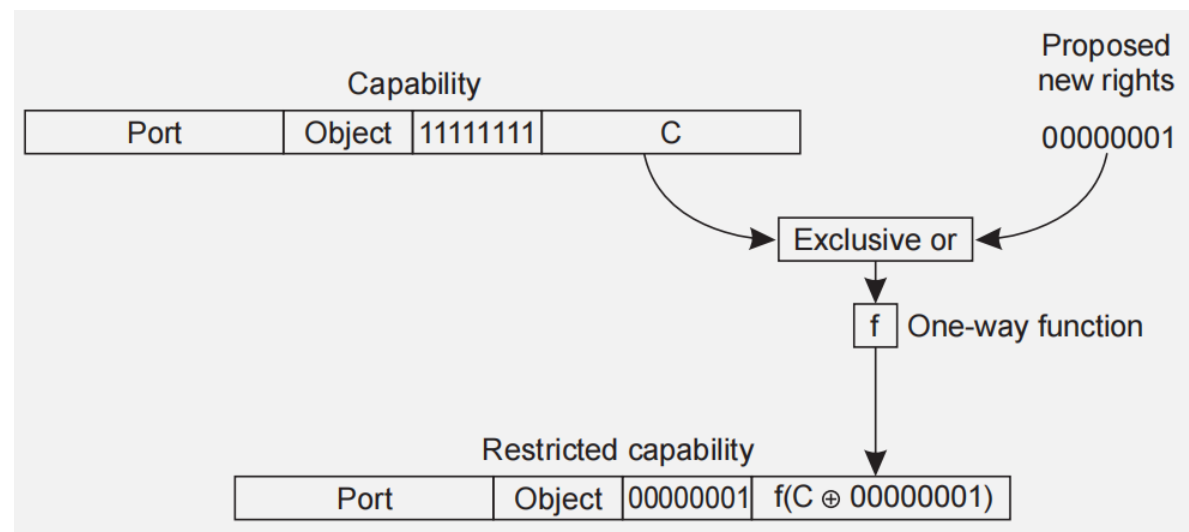
Authorization management

Issue

To avoid that each machine needs to know about all users, we use capabilities and attribute certificates to express the access rights that the holder has.

Amoeba

Restricted access rights are encoded in a capability, along with data for an integrity check to protect against tampering



安全管理

Security management

Delegation

Observation

A subject sometimes wants to delegate its privileges to an object O1, to allow that object to request services from another object O2.

Example

A client tells the print server PS to fetch a file F from the file server FS to make a hard copy $\Rightarrow$ the client delegates its read privileges on F to PS

Nonsolution

Simply hand over your attribute certificate to a delegate (which may pass it on to the next one, etc.)

安全管理

Security management

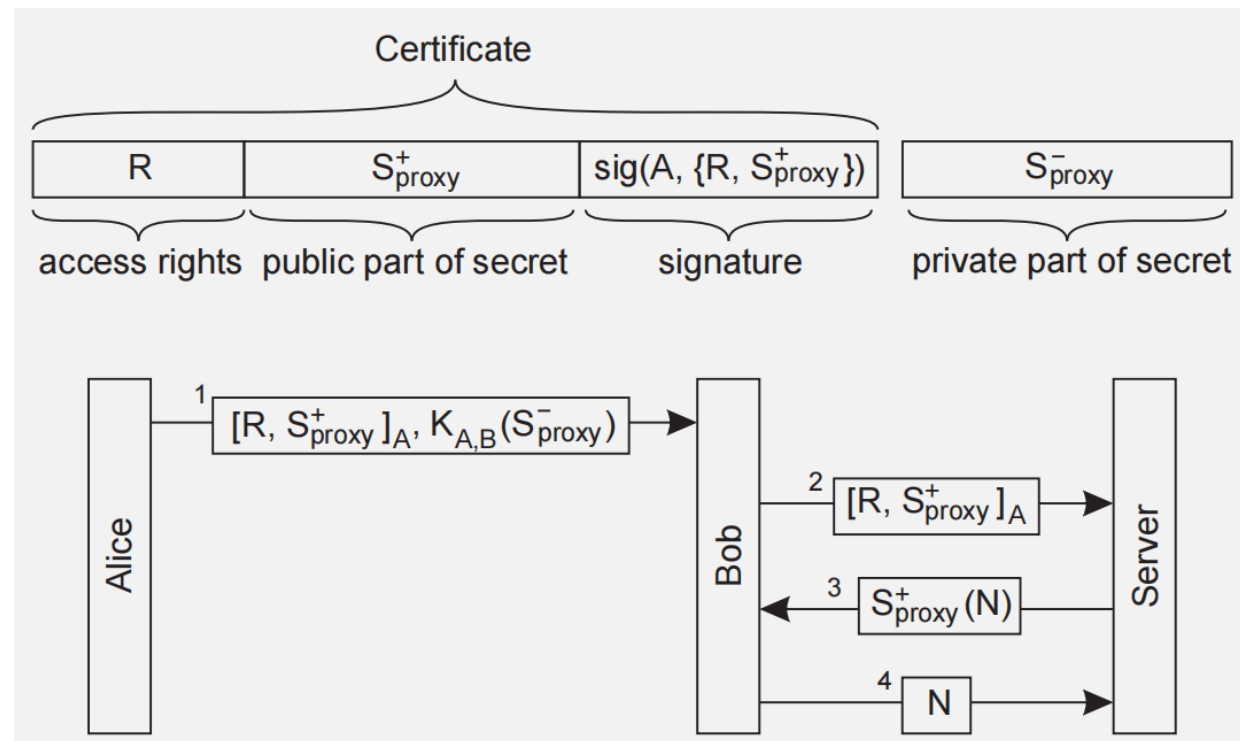
Delegate privileges

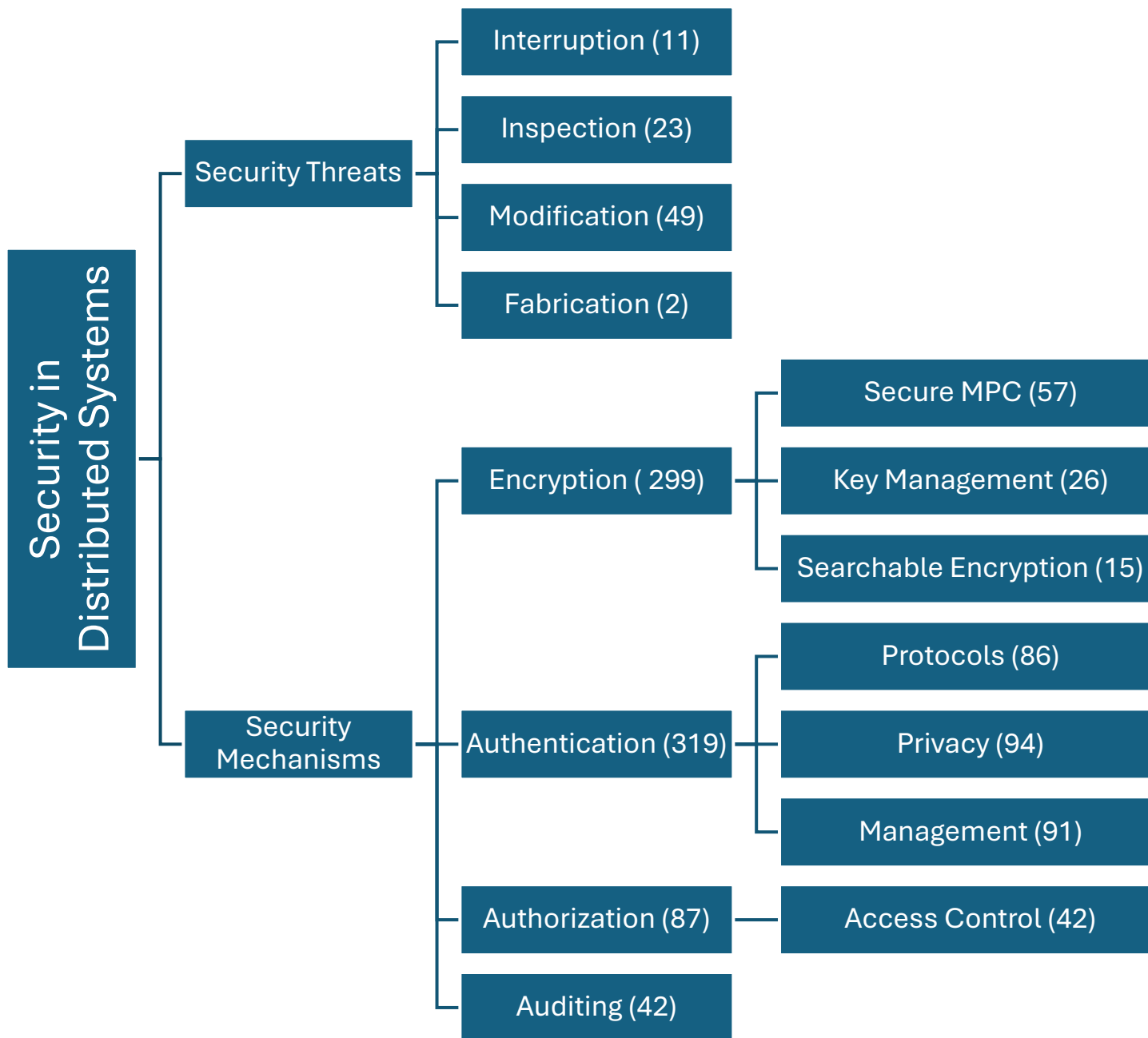
Problem

To what extent can the object trust a certificate to have originated at the initiator of the service request, without forcing the initiator to sign every certificate?

Solution

Ensure that delegation proceeds through a secure channel, and let a delegate prove it got the certificate through such a path of channels originating at the initiator.





Week 9: Security

第9周：安全

北京工业大学 分布式系统的原理

Beijing University of Technology, Principles of Distributed Systems

安全管理

Proposed Novel Research Directions

1. Secure, Verifiable Audit Trails for Edge-to-Cloud Distributed Systems

Design an end-to-end auditing framework that records and verifies events across edge devices, local gateways, and cloud services.

Use lightweight cryptographic proofs and tamper-evident logs so auditors can validate that distributed state changes happened in the claimed order.

Address resource-constrained nodes by combining secure hashing, bloom filters, and selective checkpointing.

2. Distributed Auditability for Multi-Party Federated Systems

Build a security model for auditing federated learning or multi-organization distributed computation.

Enable each party to prove compliance and data access without revealing raw inputs.

Explore zero-knowledge techniques to verify audit assertions while preserving privacy.

3. Adaptive Runtime Auditing in Microservice Meshes

Create a dynamic auditing layer that adjusts logging fidelity based on detected threat levels and service behavior.

Integrate distributed tracing data with security policies to generate audit records only when anomalies or policy violations occur.

This reduces audit overhead while preserving forensic value.

安全管理

Proposed Novel Research Directions

4. Blockchain-Inspired Audit Chains for Distributed System Configuration Changes

Propose a distributed configuration audit protocol that timestamps and chains config updates across replicated controllers. Use a permissioned consensus mechanism to ensure that each configuration change is auditable, authorized, and non-repudiable. Target systems like SDN control planes, Kubernetes clusters, or distributed storage networks.

5. Assurance of Audit Integrity under Compromised Nodes

Investigate audit schemes that remain reliable even when a subset of distributed nodes is malicious or compromised. Develop mechanisms for cross-validation of audit evidence between honest peers, and recovery of trust anchors. Consider combining quorum-based verification with hardware roots of trust.

6. Privacy-Preserving Auditing for Distributed Security Events

Design an auditing architecture that hides sensitive payloads while preserving the ability to verify security-relevant actions. Use techniques such as secure enclaves, secure multi-party computation, or differential privacy for audit log generation. Apply to systems where audit logs contain sensitive user or device metadata.

7. Automated Compliance Auditing for Distributed Access Control

Build a distributed audit system that automatically checks access control decisions across services against formal security policies. Include real-time detection of policy drift and stale authorizations. Useful for zero trust environments and large-scale distributed identity systems.